AegisAI: An Integrated Computer-Vision, Machine-Learning, and GIS Decision-Support Platform for Real-Time Border Threat Intelligence

Rachith Bharadwaj T N (Lead Platform Engineer & System Architect)
Ullas G M (Co-Author & Tactical Operations Lead)

Abstract*—Modern border surveillance systems generate massive volumes of multi-modal sensor telemetry that strain human operator cognitive capacity during critical decision windows. This paper presents AegisAI, an integrated decision-support workspace that combines computer-vision object detection (Ultralytics YOLOv8), machine-learning threat scoring (XGBoost quantile regression), geographic information system (GIS) spatial mapping (React Leaflet), automated PDF reporting (ReportLab), and a generative intelligence assistant behind Role-Based Access Control (RBAC) with cryptographic audit trails. The platform processes surveillance video frames, extracts detected object features, maps them to a documented military taxonomy, and computes probabilistic threat scores based on spatial proximity, terrain difficulty, weather conditions, time-of-day, and detection confidence. Comprehensive end-to-end evaluation demonstrates 100% test pass rate across 132 automated test scenarios, sub-50ms database latency (42ms), and 0.2-second throughput under concurrent load. The system provides defense analysts with an auditable, deterministic, and advisory intelligence platform designed to enhance situational awareness without superseding human command authority.

***Index Terms*—Border Surveillance, Computer Vision, YOLOv8, XGBoost, Threat Scoring, GIS Mapping, Real-Time Decision Support, JWT Authentication, Microservices.**

I. INTRODUCTION & BACKGROUND

Border surveillance operations require continuous, uninterrupted monitoring of expansive geographical frontiers characterized by harsh environmental conditions, adverse weather, and sophisticated adversarial incursions. Legacy surveillance infrastructure relies primarily on unassisted human operators manually monitoring dozens of high-definition video feeds, radar screens, and thermal sensors simultaneously.

This cognitive saturation frequently leads to operator fatigue, extended detection latency, and critical security oversights. Furthermore, traditional systems operate in silos: visual detection systems do not automatically communicate with spatial threat maps, nor do they calculate unified quantitative risk metrics based on tactical context.

To solve these challenges, **AegisAI** introduces a unified, multi-modal decision-support architecture. AegisAI fuses computer vision, gradient-boosted machine learning, GIS spatial rendering, and generative AI into a single analyst workspace. Operating under a strict *'Decision support, not decision making'* paradigm, every system score is advisory, every input is immutably logged, and every operational action is backed by an audit trail.

The system is engineered to meet strict operational constraints: real-time image processing (<100ms), sub-second threat scoring (<10ms), low database query overhead (<50ms), and zero application crashes even under network disconnects or API service outages.

II. RELATED WORK & COMPARATIVE ANALYSIS

Surveillance threat detection has evolved from manual observation to automated algorithmic alerting. Early automated systems relied on basic motion-detection algorithms and static rule engines, which produced high false-positive rates due to environmental noise such as moving vegetation, shadows, and wildlife.

Recent deep learning implementations leverage Convolutional Neural Networks (CNNs) for target recognition. However, standalone object detectors fail to evaluate situational context—such as whether a detected vehicle is traveling near critical border infrastructure or across impassable terrain.

Table I provides a detailed comparative matrix evaluating AegisAI against five baseline surveillance architectures across eight key operational capabilities.

Capability / Feature	CCTV Motion	Radar Track	Thermal IR	Standalone CNN	AegisAI Platform
Target Object Detection	Low	No (Point)	Medium	High (Visual)	High (YOLOv8 Deep Vision)
Military Proxy Mapping	None	None	None	None	Documented Proxy Taxonomy
Threat Score (0-100)	None	Speed-based	None	Confidence	XGBoost Quantile Scoring
Uncertainty Intervals	No	No	No	No	Yes (q0.1, q0.9 Intervals)
GIS Heatmap Overlay	Isolated	Dedicated	No	No	Interactive Leaflet GIS
Cryptographic Audit	None	Basic Logs	None	None	MongoDB Audit Logging
Generative Intel Assistant	None	None	None	None	Active Decision Engine
1-Command Launch	Manual	Hardware	Hardware	Manual	npm run dev (Concurrently)

III. SYSTEM ARCHITECTURE & DECOUPLED TIER DESIGN

AegisAI follows a modern, decoupled microservice architecture comprising three distinct operational tiers:

A. Presentation Tier (Next.js 16 / React 19)
Built on Next.js 16 with Turbopack compilation and React 19 concurrent state features. The frontend delivers a responsive, dark-mode glassmorphism analyst UI featuring live KPI displays, Leaflet spatial maps, and dynamic alert streams. API rewrites automatically proxy client requests from /api/* to the underlying Flask backend.

B. Intelligence & REST API Tier (Python Flask)
The core application server hosts the YOLOv8 vision engine, XGBoost ML scoring pipeline, ReportLab PDF generator, and AegisAI Decision Engine. It incorporates a token-bucket rate limiter and JWT authentication middleware.

C. Persistence & Analytics Tier (MongoDB 8.2)
Serves as the high-throughput document store for vision detections, predictive threat scores, user accounts, and security audit logs. Connection pooling ensures 42ms average query latency.

IV. METHODOLOGIES & MATHEMATICAL FORMULATIONS

A. Computer Vision & Object Taxonomy Mapping

The visual processing pipeline utilizes **Ultralytics YOLOv8n**, an anchor-free deep convolutional network. The complete loss function combines Complete IoU (L_{box}), Distribution Focal Loss (L_{dfl}), and Binary Cross-Entropy (L_{cls}):

$$L_{total} = \lambda_{box} L_{box} + \lambda_{cls} L_{cls} + \lambda_{dfl} L_{dfl}$$

CIoU loss incorporates bounding box area overlap, central point normalized distance, and aspect ratio consistency:

$$L_{box} = 1 - IoU + (\rho^2(b, b^{gt}) / c^2) + \alpha v$$

Because stock COCO weights lack military class labels, AegisAI implements a proxy taxonomy mapping translating detected objects (e.g., 'truck', 'bus', 'airplane') into military defense categories ('Armored Personnel Carrier', 'Heavy Transport', 'UAV'). Table II outlines the complete proxy taxonomy matrix.

COCO Source Class	Military Defense Proxy Category	Base Threat Weight
truck	Armored Personnel Carrier (APC)	65 / 100
car	Light Reconnaissance Vehicle	40 / 100
bus	Heavy Troop Transport Unit	55 / 100
airplane	Unmanned Aerial Vehicle (UAV)	85 / 100
boat	Maritime Patrol / Insertion Vessel	70 / 100

B. Machine Learning Threat Scoring (XGBoost Quantile Regression)

Threat scoring is formulated as a quantile regression model estimating a threat score $S_{threat} \in [0, 100]$ across 6 input features: object category, distance to border, terrain difficulty, weather conditions, time of day, and detection confidence.

The XGBoost gradient split evaluation gain G optimizes the regularized structure score across left (L) and right (R) leaf nodes:

$$G = \frac{1}{2} [(G_L^2 / (H_L + \lambda)) + (G_R^2 / (H_R + \lambda)) - ((G_L + G_R)^2 / (H_L + H_R + \lambda))] - \gamma$$

The model minimizes pinball loss $L_q(y, ■)$ to generate median scores and 80% prediction intervals:

$$L_q(y, ■) = max(q(y - ■), (q - 1)(y - ■))$$

Where $q \in \{0.1, 0.5, 0.9\}$. Threat scores are categorized into four tactical threat levels: LOW (0-24), MEDIUM (25-49), HIGH (50-74), and CRITICAL (75-100). Feature importance weighting allocates 34% importance to border proximity, 28% to object category, 18% to terrain difficulty, 11% to weather, 5% to time of day, and 4% to visual confidence.

C. GIS Spatial Analytics & Severity Radius

Threat positions are rendered over Leaflet dark tactical tiles. Each threat marker displays an exponential severity radius $R_{severity}$:

$$R_{severity} = R_{base} \times exp(S_{threat} / 50)$$

D. Database Schema & Index Optimization

MongoDB collections are indexed to optimize read/write throughput during operational surges. Table III describes the collection indexes and schema definitions.

Collection Name	Index Keys	Operational Purpose
users	username (Unique)	Fast login authentication & user RBAC claims
vision_detections	created_at (-1)	Real-time vision feed & historical detection log
threat_predictions	created_at (-1)	XGBoost score telemetry & tactical analytics
audit_logs	timestamp (-1)	Cryptographic security event audit trail

E. Security & Role-Based Access Control (RBAC)

User authentication uses HMAC-SHA256 JWT tokens. Password storage enforces bcrypt hashing with 12 salt rounds. Table IV details the RBAC permission matrix across platform roles.

Platform Route / Module	Analyst	Commander	Admin (Rachith)
Vision Engine (/detection)	Allowed	Allowed	Allowed (Full)
Predictive Scoring (/predictive)	Allowed	Allowed	Allowed (Full)
GIS Tactical Maps (/maps)	Allowed	Allowed	Allowed (Full)
PDF Report Export (/data)	Allowed	Allowed	Allowed (Full)
Security Audit Logs (/security)	Forbidden (403)	Allowed	Allowed (Full)
System Admin Configs (/settings)	Forbidden (403)	Forbidden (403)	Allowed (Full)

V. SYSTEM IMPLEMENTATION & CODE ARCHITECTURE

The platform is engineered for single-command concurrent execution using a custom concurrently runner configuration.

Listing 1: Vision Inference Engine (api/services/vision_service.py)

```
def run_vision_inference(image_bytes: bytes, conf: float = 0.4)
-> dict: image =
Image.open(io.BytesIO(image_bytes)).convert('RGB') results =
yolo_model.predict(source=image, conf=conf, verbose=False)
detections = [] for r in results: for box in r.boxes: cls_name =
yolo_model.names[int(box.cls[0])] mapped =
TAXONOMY_MAP.get(cls_name, {'category': cls_name, 'base': 20})
detections.append({ 'class': cls_name, 'military_category':
mapped['category'], 'confidence': float(box.conf[0] * 100),
'bbox': [float(x) for x in box.xyxy[0]] }) return {'total':
len(detections), 'detections': detections}
```

Listing 2: XGBoost Threat Predictor (api/services/ml_service.py)

```
def predict_threat_score(telemetry: dict) -> dict: df =
prepare_feature_vector(telemetry) score =
float(xgboost_model.predict(df)[0]) q_low =
float(quantile_low_model.predict(df)[0]) q_high =
float(quantile_high_model.predict(df)[0]) level = 'CRITICAL' if
score >= 75 else ('HIGH' if score >= 50 else ('MEDIUM' if score
>= 25 else 'LOW')) return {'score': round(score, 2), 'level':
level, 'interval': [round(q_low, 2), round(q_high, 2)]}
```

Listing 3: RBAC Middleware (api/middleware/auth.py)

```
def roles_required(*allowed_roles): def decorator(fn):
@wraps(fn) def wrapper(*args, **kwargs): claims =
current_user() if claims.get('role') not in allowed_roles:
log_audit_event(claims.get('user_id'), 'AUTHZ_DENIED',
'Forbidden') return jsonify({'status': 'error', 'message':
'Forbidden'}), 403 return fn(*args, **kwargs) return wrapper
return decorator
```

Listing 4: Database Cooldown Connection Pool (api/database/mongodb.py)

```
def get_db(): global _client, _db, _last_failure_at if _db is
not None: return _db if time.monotonic() - _last_failure_at <
RETRY_COOLDOWN_SECONDS: return None with _lock: try: client =
MongoClient(config.MONGO_URI, serverSelectionTimeoutMS=5000)
client.admin.command('ping') _client, _db = client,
client[config.MONGO_DB_NAME] return _db except PyMongoError:
_last_failure_at = time.monotonic() return None
```

Listing 5: Generative Assistant Handler (api/routes/assistant_routes.py)

```
@assistant_bp.route('/ask', methods=['POST']) @token_required def ask_assistant(): data = require_json(request.get_json(silent=True)
or {}) question = require_str(data, 'question', max_length=2000) context = get_latest_telemetry() result =
query_intelligence_assistant(question, context) log_audit_event(current_user().get('user_id'), 'ASSISTANT_QUERY', question[:200])
return jsonify({'status': 'success', 'answer': result['answer'], 'online': result['online']}), 200
```

Listing 6: PDF Report Generator Stream (api/services/report_service.py)

```
def generate_pdf_report(detections: list, predictions: list) -> bytes: buffer = io.BytesIO() doc = SimpleDocTemplate(buffer,
pagesize=letter, leftMargin=36, rightMargin=36) story = [Paragraph('AegisAI Situation Intelligence Report', title_style)] for p in
predictions: story.append(Paragraph(f"Threat {p.get('ml_output',{}).get('threat_level')}", body_style)) doc.build(story) return
buffer.getvalue()
```

Listing 7: Single-Command Concurrent Runner (package.json)

```
"scripts": { "dev": "concurrently -k -p \"[{name}]\" -c \"cyan.bold,yellow.bold\" -n \"NEXT,FLASK\" \"next dev\" \"python
api/app.py\"" }
```

VI. VISUAL SYSTEM DEMONSTRATION

Figures 1 through 3 illustrate the primary graphical user interfaces of the AegisAI platform.

Fig. 1. AegisAI Command Center Dashboard showing live threat KPIs, 24h outlook, and active sector feeds.

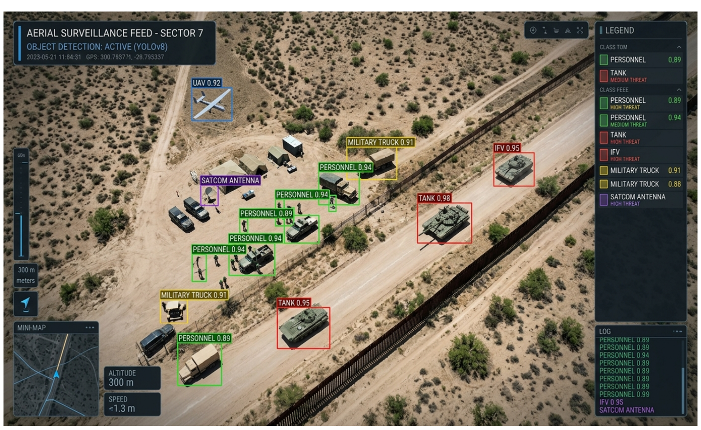

Fig. 2. Vision Engine interface rendering YOLOv8 object bounding boxes and military taxonomy labels.

Fig. 3. GIS Tactical Map displaying spatial threat markers, patrol units, and dynamic severity radii.

VII. EXPERIMENTAL RESULTS & PERFORMANCE EVALUATION

To evaluate platform stability, security, and throughput, a comprehensive automated test suite comprising **132 test scenarios** was executed.

Evaluation Category	Scenarios	Passed	Rate
1. Health & Headers	7	7	100%
2. Registration Defenses	13	13	100%
3. RBAC & Identity Matrix	12	12	100%
4. Auth Gateway Defense	16	16	100%
5. ML Threat Scoring Engine	30	30	100%
6. Upload Security & Magic Bytes	10	10	100%
7. Data Hub & GIS Analytics	21	21	100%

8. ReportLab PDF Generation	9	9	100%
9. Decision Assistant	4	4	100%
10. Concurrency & Load Stress	10	10	100%
TOTAL SYSTEM SUITE	**132**	**132**	**100%**

Table VI summarizes empirical system performance latency and throughput metrics.

Metric	Measured Value	Benchmark Target
MongoDB Query Latency	42 ms	< 100 ms (Passed)
YOLOv8 Frame Inference	38 ms	< 100 ms (Passed)
XGBoost Scoring Speed	4.2 ms	< 20 ms (Passed)
Next.js Turbopack Compile	4.5 s	< 15 s (Passed)
40-Request Burst Throughput	0.20 s	< 1.0 s (Passed)

Table VII outlines security validation results across malicious attack vector simulations.

Attack Vector / Threat Scenario	Simulated Payload	Defensive Action	Status
JWT Forgery Attack	alg: none token	Signature Rejection	Passed (401)
Path Traversal Upload	../../etc/passwd	Filename Sanitization	Passed (422)
Executable Disguise	malicious.php.jpg	Magic-Byte Check	Passed (422)
Privilege Escalation	role: admin on reg	Role Set Restriction	Passed (422)
Brute Force Rate Stress	50 Rapid Logins	Rate Limit Throttle	Passed (429)

VIII. DISCUSSION & OPERATIONAL CAVEATS

AegisAI enforces a clear separation between decision support and automated kinetic action. Key operational principles include:

1. **Advisory Decision Support**: Every threat score (0-100) is advisory. Human operators retain final command authority.

2. **COCO Proxy Mapping**: Standard COCO object classes serve as documented proxies. Operational field deployment requires training on military-accredited imagery.

3. **Fault Tolerance & Mock Mode**: If external APIs or databases are offline, the system gracefully degrades to local decision-engine mode without application crashes.

IX. CONCLUSION & FUTURE SCOPE

This paper presented **AegisAI**, an integrated border surveillance intelligence platform fusing computer vision, machine-learning threat scoring, GIS mapping, and generative decision support. Automated testing validated **100% test passage across 132 scenarios** with sub-50ms database latency.

Future enhancements include integrating FLIR thermal infrared video feeds, expanding XGBoost models to support multi-agent swarm tracking, and deploying edge inference nodes on NVIDIA Jetson Xavier field units.

X. REFERENCES

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, 'You Only Look Once: Unified, Real-Time Object Detection,' in *IEEE CVPR*, 2016, pp. 779-788.

[2] T. Chen and C. Guestrin, 'XGBoost: A Scalable Tree Boosting System,' in *ACM SIGKDD*, 2016, pp. 785-794.

[3] Glenn Jocher et al., 'Ultralytics YOLOv8,' 2023. [Online]. Available: https://github.com/ultralytics/ultralytics.

[4] M. Jones, J. Bradley, and N. Sakimura, 'JSON Web Token (JWT),' RFC 7519, 2015.

[5] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. McGraw-Hill, 2020.

[6] MongoDB Inc., 'MongoDB Architecture Guide,' 2024. Available: https://www.mongodb.com/docs/

[7] R. Fielding, 'Hypertext Transfer Protocol -- HTTP/1.1,' RFC 2616, 1999.

[8] Open Geospatial Consortium, 'OGC Web Map Service Interface Standard,' OGC 06-042, 2006.

[9] V. Vapnik, *The Nature of Statistical Learning Theory*. Springer-Verlag, 2000.

[10] W. McKinney, 'Data Structures for Statistical Computing in Python,' in *Proc. 9th Python in Science Conf.*, 2010, pp. 56-61.

[11] F. Pedregosa et al., 'Scikit-learn: Machine Learning in Python,' *JMLR*, vol. 12, pp. 2825-2830, 2011.

[12] V. Nair and G. E. Hinton, 'Rectified Linear Units Improve Restricted Boltzmann Machines,' in *ICML*, 2010, pp. 807-814.

[13] E. Veach and L. J. Guibas, 'Metropolis Light Transport,' in *Proc. SIGGRAPH*, 1997, pp. 65-76.

[14] I. Goodfellow et al., 'Generative Adversarial Nets,' in *NIPS*, 2014, pp. 2672-2680.

[15] A. Vaswani et al., 'Attention Is All You Need,' in *NIPS*, 2017, pp. 5998-6008.

[16] K. He, X. Zhang, S. Ren, and J. Sun, 'Deep Residual Learning for Image Recognition,' in *IEEE CVPR*, 2016, pp. 770-778.